CONFIDENCE-AWARE, HEALTH-ADAPTIVE RUNTIME ASSURANCE FOR UNCREWED AIRCRAFT

An Integrated Architecture for Safety Gating, Mission-State Reconciliation, and Evidence Logging

Research article type: Design-science architecture paper and proposed verification protocol | Version 1.0 | 15 October 2026

ABSTRACT

Autonomous uncrewed aircraft must reconcile uncertain sensor observations, evolving vehicle health, mission constraints, remote authority, and real-time control. This paper develops a research formulation of an integrated architecture described in a supplied technical and patent-drafting package. The architecture associates fused state estimates with confidence and provenance; maintains a distinct uncertain environment model; generates and forward-evaluates candidate trajectories; and places a deterministic safety envelope between AI-assisted planning and flight control. Its additional cross-layer mechanisms update planning constraints in response to predicted degradation, authenticate supervisory inputs, hold unresolved local/remote mission-state conflicts in a non-executable state, and record decision evidence with optional cryptographic integrity protection. A design-science method is used to reconstruct the architecture, define interface invariants, and propose testable hypotheses and a staged verification protocol. Formal notation is introduced for health-conditioned feasible trajectory sets, accept/modify/reject gating, authority reconciliation, and chained evidence records. The proposed evaluation includes simulation, software-in-the-loop, hardware-in-the-loop, and only subsequently approved flight testing, with fault injection for sensor degradation, health changes, communication loss, stale commands, and logging faults. The supplied package contains no source code, benchmark dataset, simulation output, or flight-test measurements; therefore, this paper reports no empirical performance result and makes no claim of certification or demonstrated safety. Its contribution is a traceable research specification for validating whether the integrated architecture can improve constraint enforcement, degraded-mode behavior, and post-event reconstructability.

Keywords: uncrewed aircraft; runtime assurance; safety filter; sensor fusion; uncertainty; health-aware planning; command authority; mission reconciliation; evidence logging; verification and validation

EVIDENCE STATUS

This is a research draft derived from the supplied UCAS patent-application assembly package. Architecture elements are source-supported descriptions; equations, invariants, hypotheses, and the test plan are research formalizations proposed here. No experimental results are represented as having been obtained.

1. Introduction

Uncrewed aircraft increasingly combine heterogeneous sensing, onboard computation, optimization or learned planning, remote supervision, and feedback control. The resulting system-level risk is not limited to an inaccurate perception model or a poor trajectory. It also arises when uncertainty is lost between modules,

a degradation is detected too late to influence planning, a remote command is accepted without current authority, a conflict between local and remote state becomes executable, or an incident cannot be reconstructed from retained records. Multisensor fusion has a long history in estimation and autonomous-vehicle guidance [1], while more recent safety-critical autonomy research has emphasized runtime assurance (RTA), supervisory switching, and safety filters [2–5].

The supplied UCAS package describes a combined architecture with nine linked concerns: heterogeneous sensing; confidence- and provenance-aware state estimation; an environment model that represents uncertainty separately; mission-contract handling; AI candidate generation and forward evaluation; deterministic accept/modify/reject gating; health-aware constraint updates; authenticated communications and mission-state reconciliation; and evidence logging. The central design proposition is that safety-relevant context should affect what is allowed to execute, not merely be recorded after the fact.

This paper turns that proposal into a research specification. It does not assert that the system is implemented, novel under patent law, or flight-safe. Instead, it asks whether the proposed interfaces and control relationships can be made precise, verified, and measured. The article makes four contributions: (1) a layered architecture and dependency model; (2) candidate formal invariants for safety gating, health-conditioned planning, command authority, and evidence integrity; (3) explicit hypotheses and measurable outcomes; and (4) a staged validation plan that begins in simulation and permits flight trials only after prior safety and assurance gates have been satisfied.

2. Background and related work

2.1 Heterogeneous sensing and uncertainty

Multisensor data fusion combines complementary and redundant observations to estimate state or environmental features [1]. In an aircraft architecture, a fused estimate should not be treated as equally reliable in every dimension or at every instant. Sensor age, calibration, geometry, occlusion, environmental conditions, and disagreement between sources can change the effective quality of an estimate. A useful research distinction is therefore between the estimated state, a confidence or uncertainty representation associated with that estimate, and provenance that records where the information came from and how it was transformed. Confidence scores should not be interpreted as calibrated probabilities unless they have been calibrated and validated for the relevant operating domain.

2.2 Runtime assurance and safety filtering

RTA architectures seek to constrain the behavior of complex or insufficiently assured functions through monitoring and a trusted intervention path. Schierman et al. discuss RTA for autonomous aerospace systems, including interacting assurance layers across guidance, flight management, and mission planning [2]. Slagel et al. formalize a Simplex-style RTA framework for UAS using theorem-proving techniques and describe the importance of monitor sampling assumptions [3]. NASA has also reported small-UAS testing of runtime assurance combined with risk-mitigation capabilities, including flight scenarios with sequential and simultaneous hazards [4]. These results establish relevant precedent for assurance architectures, but they do not validate the particular integrated design analyzed here.

Safety-filter research similarly studies how to combine high-performance learned or data-driven commands with a safety mechanism that intervenes when necessary [5]. ASTM F3269-21 provides a standard practice for developing and evaluating RTA architectures intended to bound aircraft behavior containing complex functions [6]. The UCAS design aligns with this general pattern by separating candidate generation from deterministic safety enforcement. Its proposed distinction is the integration of the safety gate with health-

conditioned planning, authority state, local/remote reconciliation, and decision evidence; the effectiveness of that integration remains an empirical question.

2.3 AI risk, cybersecurity, and auditability

The NIST AI Risk Management Framework organizes risk work around GOVERN, MAP, MEASURE, and MANAGE, with attention to validity, reliability, safety, security, resilience, accountability, and transparency [7]. These dimensions support a lifecycle approach to evaluation rather than treating an AI planner as an isolated model. NIST SP 800-193 addresses protection, detection, and recovery mechanisms for platform firmware and critical data [8], while SP 800-92 provides broader guidance on security log-management processes [9]. This paper applies those principles at the architectural level: authenticate the authority of mission updates, track model identity and command provenance, retain a structured explanation of gate outcomes, and protect evidence records against undetected alteration. A cryptographic log can help detect tampering; it does not prove that the recorded sensor values were truthful, that the logger was uncompromised at capture time, or that the records remain available.

2.4 Research gap and framing

The specific research question is not whether safety gating, health monitoring, authenticated commands, or logging are individually useful. It is whether a coherent set of interfaces can preserve safety-relevant dependencies across these functions and whether that coherence produces measurable benefits under faults. This paper treats the integrated architecture as a hypothesis-bearing design, not as a proven state of the art or a claim of exclusive novelty. Any assertion of technical novelty, patentability, standards conformity, or regulatory acceptability requires separate evidence and review.

3. Materials and methods

3.1 Source materials

The primary source was the user-provided UCAS_FINAL_Patent_Application_Package.zip. The package contains a consolidated U.S. utility-patent specification and claims, an invention-disclosure/attorney-review memorandum, a filing checklist, a corrected six-figure drawing set, and earlier source drawings and images retained for provenance. The principal design description and reference numerals were read from the specification and cross-checked against the system overview and planning/safety drawings. The package itself describes a finalized drafting assembly, not a filed application or an experimentally validated system.

The analysis used qualitative document synthesis. Architecture statements were extracted, grouped by information dependency, and recast into functional modules. A second pass identified the safety, authority, health, timing, and evidence conditions that would need to be explicit for verification. The literature review was targeted to multisensor fusion, runtime assurance, safety filters, AI risk management, platform resiliency, and security logging [1–9].

3.2 Evidence classification

Evidence class	Meaning in this paper	Examples
Source-supported architecture	Explicitly represented in the supplied specification or diagrams.	Fused state with confidence/provenance; forward evaluation; deterministic gate; health model; mission reconciliation; evidence logger.
Research formalization	Notation or interface invariants introduced here to make the proposed design testable.	Health-conditioned feasible set; trajectory digest/version binding; formal non-executability of unresolved conflicts.

Evidence class	Meaning in this paper	Examples
Evaluation proposal	A suggested method, metric, or acceptance condition that has not yet been run.	Fault-injection matrix; latency percentiles; violation rate; ablation tests; statistical analysis plan.
Unverified implementation detail	A value or mechanism not established by the supplied package.	Actual thresholds, code, hardware selection, timing budget, aircraft performance envelope, cryptographic deployment, or measured reliability.

No aircraft, test bench, flight data, model weights, source code, or performance dataset was included in the supplied material. Consequently, no hypothesis is described as confirmed, no effect size is reported, and no flight-readiness inference is made.

3.3 System boundary and assumptions

The system boundary includes the aircraft-side functions that transform observations and authorized mission inputs into executable trajectory commands, plus the logging path that records decision context. Sensors, aircraft dynamics, environmental truth, ground-supervisor systems, cryptographic key management, and the flight-test range are external dependencies whose behavior must be modeled or specified. It is assumed that each deployed component exposes timestamps and identity metadata where required, that the flight controller can enforce the interface contract, and that a safety authority defines the applicable hard constraints and fallback behavior before testing. These are design assumptions to verify, not facts established by the package.

4. Proposed architecture

The architecture is organized around a nominal decision path and cross-cutting supervisory paths. Sensor data are fused into an aircraft state, an environment model represents uncertainty, and a mission contract encodes objectives and permitted actions. The planner creates candidate trajectories rather than directly driving actuators. A forward evaluator predicts candidate consequences, after which a deterministic safety envelope applies hard constraints. Health, communication, authority, and mission reconciliation modify the feasible action set and/or block execution when prerequisites are not satisfied. An evidence logger records the context needed to review a decision after execution.

FIG. 5 SYSTEM OVERVIEW — AUTONOMY, SAFETY, RESILIENCE, AND EVIDENCE

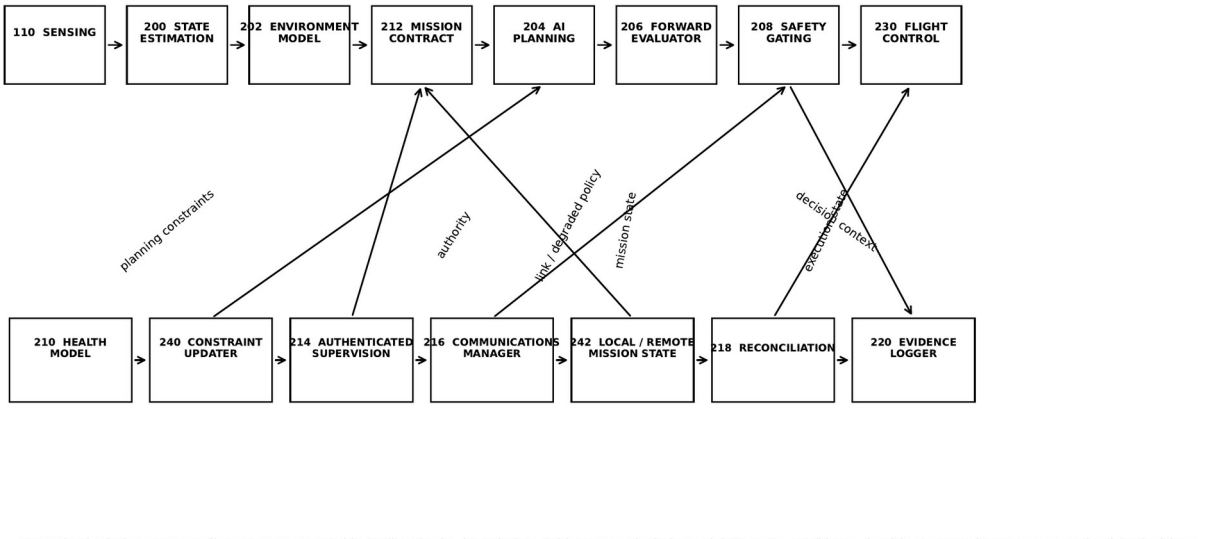


Figure 1. System overview and cross-layer dependencies. Source: supplied UCAS package, FIG. 5 (corrected drawing set).

4.1 Functional modules and interfaces

Module	Required input(s)	Output / contract	Safety significance
Heterogeneous sensing	Sensor observations, timestamps, calibration/status.	Time-indexed observations with source identity.	Detect stale, missing, inconsistent, or degraded sources.
State estimation	Synchronized sensor observations.	Fused state plus per-field confidence/uncertainty and provenance.	Keep quality metadata attached downstream; avoid false precision.
Environment model	Perception, map, navigation and external environment inputs.	Environment representation with explicit uncertainty.	Bound unknowns and avoid treating uncertain environment data as fact.
Mission contract	Authorized mission objectives, constraints, timing, permissions.	Versioned and authority-scoped mission state.	Constrain plan generation and accepted remote updates.
AI planner + forward evaluator	State, environment, mission, current health-conditioned constraints.	Candidate trajectories and predicted responses.	Candidate generation is not execution authority.
Deterministic safety envelope	Candidate, predicted response, constraint version, authority state.	ACCEPT, MODIFY or REJECT plus reason code.	No trajectory reaches control without current gate approval.
Health model + constraint updater	Telemetry, fault flags, trend estimates, resource status.	Health state and revised planning constraints.	Move degradation into planning feasibility before execution.
Communication + reconciliation	Authenticated remote messages, local state, sequence/time/version metadata.	Authorized state transition or explicit non-executable conflict.	Reject stale/unauthorized updates; apply a defined degraded policy.
Evidence logger	Model ID, state, confidence, environment uncertainty, health, commands, gate outcome.	Ordered records with integrity metadata.	Enable reconstruction and detect some forms of record alteration.

4.2 Confidence- and provenance-aware state estimation

Let the state estimator fuse measurements from m sensor sources. For each update, the output should include a state estimate, a quality representation, and provenance. Confidence may be represented as covariance, interval width, a calibrated probability, a quality score, or a vector of per-variable values; the chosen semantics must be explicit. Provenance may include sensor/source identifier, timestamp, calibration state, processing version, and model version. The architecture should preserve the distinction between epistemic/measurement uncertainty in the state estimate and uncertainty in the environment model. Those quantities can interact during planning, but they should not be collapsed into a single undocumented scalar.

A key implementation requirement is temporal and semantic alignment. Before a planner consumes a fused state, the interface should define timestamp age bounds, frame conventions, units, missing-field semantics, and what constitutes invalid or uncalibrated confidence. If observations are inconsistent beyond a specified rule, the estimator should expose that condition rather than silently averaging the conflict. The precise filter or fusion algorithm is intentionally left open by the source package and must be selected and evaluated for the vehicle, sensor suite, and operational domain.

4.3 Mission contracts and environment uncertainty

The environment model can represent terrain, weather, obstacles, traffic, hazards, and navigation information, with uncertainty encoded in a form appropriate to each data source. The mission contract is a distinct structured object representing objectives, constraints, permissions, sequence, timing, authority, and permitted update rules. Keeping these objects distinct enables testable questions: whether a planner used the correct mission version; whether uncertainty was propagated into trajectory evaluation; and whether an authorized mission revision changed only the fields that the signer or authority policy permitted it to change.

A mission contract should carry an identifier, version, issuing authority, issue time, validity interval or expiry policy, and integrity/authentication status. Fields that change execution authority require stronger transition rules than ordinary mission metadata. The schema, cryptographic mechanisms, and trust-root lifecycle remain implementation choices and are not specified in sufficient detail in the supplied package to evaluate here.

4.4 Candidate generation, forward evaluation, and deterministic gating

The planner proposes one or more candidate trajectories. A forward evaluator predicts the vehicle response to each candidate under the current state, environment representation, and health-conditioned limits. The gate then evaluates both the candidate and the predicted response against hard constraints. The source design defines three outcomes: ACCEPT, MODIFY, or REJECT. The distinction is meaningful only if the downstream contract is precise: a modification creates a new candidate and must be re-evaluated; a rejection must not be converted into execution by a later module; and a fallback action must have its own explicit authorization and assurance basis.

FIG. 4 AI PLANNING WITH DETERMINISTIC SAFETY GATING AND HEALTH-AWARE RECONFIGURATION

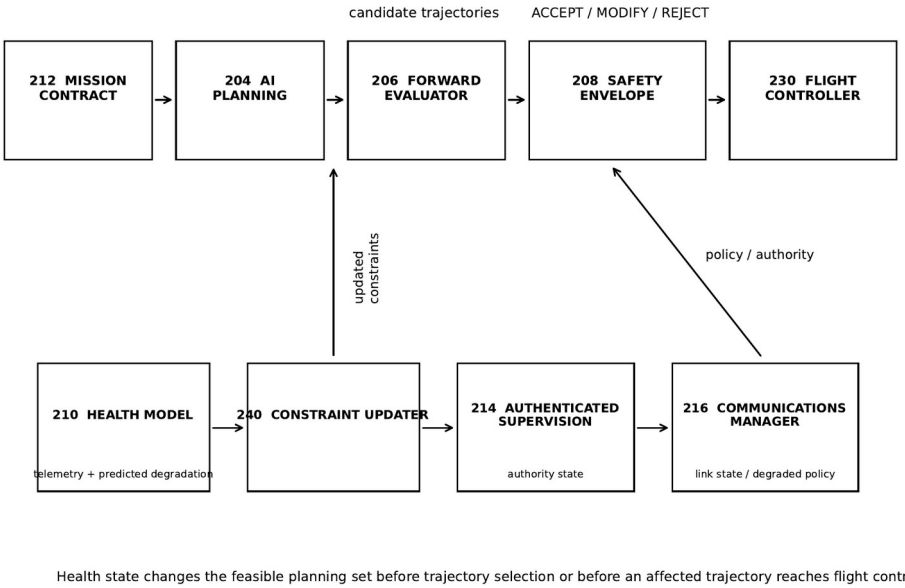


Figure 2. Candidate trajectory evaluation, deterministic gating, and health-aware constraint updates. Source: supplied UCAS package, FIG. 4 (corrected drawing set).

The gate should be implemented in a sufficiently isolated execution domain that a failure or malformed output from the planner cannot bypass it. Logical separation may be possible on shared hardware, but then access control, scheduling interference, fault containment, and failure propagation need evidence. The source package permits shared or distributed computing, so physical independence cannot be assumed merely from the block diagram.

4.5 Health-aware constraint updates

A health model consumes telemetry and may identify or predict degradation in sensing, compute, communication, power, propulsion, actuators, or other vehicle resources. Its safety-relevant output is not just a diagnostic alert; it changes one or more planning constraints before candidate selection or before an affected trajectory reaches the controller. Examples include limits on maneuvering, actuator authority, allowable sensor reliance, communication assumptions, endurance, or compute load. The architecture therefore conditions the feasible trajectory set on health state.

The principal validation concern is the detection-to-action chain. Tests must establish when degradation becomes observable, how uncertainty in a health prediction is represented, how constraints are updated atomically, and what happens if the health estimator is late, inconsistent, or unavailable. Conservative reconfiguration can prevent infeasible plans but can also create excessive rejection, loss of mission utility, or oscillation between constraint sets. Those trade-offs must be measured rather than presumed beneficial.

4.6 Authenticated supervision, communications degradation, and reconciliation

Remote supervision and communications are treated as explicit authority inputs. A message may be authentic but stale, or recent but outside the sender's authority; the receiving system therefore needs both integrity/authentication checks and semantic authorization checks. Link availability and link integrity are

separate conditions. When a defined condition fails, a degraded policy may limit the vehicle to a pre-authorized action set or allow continuation under already-authenticated mission constraints while refusing unauthenticated state changes.

The reconciliation engine compares local and remote mission state using metadata such as sequence number, time, authority state, mission version, and execution status. An authenticated revocation may supersede a stale command when the policy gives it precedence. A conflict whose precedence cannot be resolved should remain explicitly non-executable until an authorized resolution exists. This is stronger than merely logging a conflict, because the execution path must consume the conflict state as a blocking condition.

4.7 Evidence logging and integrity

For each consequential decision, an evidence record can contain the model identifier/version, configuration or checksum, state snapshot, confidence information, environment uncertainty, health state, local and remote mission versions, command provenance, communications status, gate outcome, reason code, and execution status. The source package proposes secure storage and integrity mechanisms such as hash chaining, digital signatures, or authenticated append-only storage. A chained record can be represented as an event payload plus a digest linking it to the prior record. This can support detection of alteration or sequence disruption, but it does not by itself guarantee completeness, trustworthy capture, time accuracy, secure key custody, or durable storage.

The logging system is part of the real-time system and can itself fail. The implementation must specify whether logging is synchronous or buffered, how queue overflow is handled, which fields are mandatory, how timestamps are protected, how key rotation is recorded, and how records are recovered after power loss. A safety-critical decision should not silently claim a fully auditable record when the record was dropped or integrity could not be verified. NIST log-management and platform-resiliency guidance provide useful lifecycle references for this work [8,9].

5. Formal model and candidate invariants

The following mathematical formulation is introduced to make the architecture amenable to model checking, property-based testing, and implementation-level assertions. It is not copied as an already-implemented algorithm from the supplied package. Let t be a discrete decision epoch and let the system state be:

$$z_t = (\hat{x}_t, q_t, p_t, E_t, M_t, H_t, A_t, L_t, C_t)$$

where $\hat{x}_t$ is the fused aircraft state; q_t is confidence/uncertainty; p_t is information provenance; E_t is the uncertain environment model; M_t is the active mission contract; H_t is vehicle health; A_t is authority state; L_t is communication/link state; and C_t is the reconciled mission state, including any unresolved conflict. Define Γ_t as the planner's candidate set.

$$\Gamma_t = \Pi(\hat{x}_t, q_t, E_t, M_t, H_t, A_t)$$

For a candidate trajectory γ and predictor G, the predicted response over horizon T is:

$$\tilde{x}_{t:t+T} = G(\hat{x}_t, \gamma, E_t, H_t, M_t)$$

Let $g_j(\cdot)$ represent hard safety or mission constraints, where the sign convention is $g_j \leq 0$ for satisfaction. The health-conditioned feasible set is:

$$F_t = \{\gamma \in \Gamma_t : g_j(\tilde{x}_{t:t+T}, H_t, M_t) \leq 0 \text{ for every required constraint } j\}$$

This set may also include constraints for geofencing, resource bounds, authority, timing, communications, and uncertainty margins defined by the system safety requirements. If no candidate belongs to F_t, the system

must not silently select the least-unsafe nominal candidate and label it accepted; it must invoke an explicitly defined and authorized contingency or remain in a controlled non-execution state.

5.1 Gate semantics

The safety decision is a function of the candidate, predicted response, active constraint version, and current authority/mission state. The proposed semantics are:

$$D_t(\gamma) = \text{ACCEPT, if } \gamma \text{ is feasible and all required checks pass}$$

$$D_t(\gamma) = \text{MODIFY, if a modified } \gamma' \text{ can be made feasible and revalidated}$$

$$D_t(\gamma) = \text{REJECT, otherwise}$$

For MODIFY, a candidate implementation can solve a constrained projection problem, minimizing deviation from the planner proposal subject to hard constraints. This is a design option, not a required choice of algorithm:

$$\gamma' = \arg\min_u \; J(u, \gamma) \quad \text{subject to} \quad g_j(G(\hat{x}_t, u, H_t, E_t), H_t, M_t) \le 0$$

Any modified result must be evaluated against the same or a newer compatible constraint version before reaching the flight controller. A candidate accepted against one state or constraint version must not be reused after a relevant state, mission, health, or authority update without revalidation. The execution interface should bind the approval to a trajectory digest, state/constraint version, and validity interval.

5.2 Candidate safety and authority properties

ID	Candidate invariant	Verification approach
INV-1: Gate-before-control	Every trajectory received by flight control carries an ACCEPT or revalidated MODIFY decision bound to the same trajectory digest and applicable constraint version.	Interface assertions, runtime monitor, negative tests for stale approvals, and model checking of transition logic.
INV-2: Unresolved conflicts block execution	If mission-state conflict C_t is unresolved, no new remote-derived mission state or affected candidate is executable except a separately authorized fallback.	State-machine model checking and fuzzed local/remote version/authority sequences.
INV-3: Health-conditioned feasibility	Any candidate selected after a valid health update is checked against the corresponding updated constraints.	Fault-injection timing tests and assertions across asynchronous health/planner updates.
INV-4: Authority checked before mutation	A remote update cannot mutate execution-relevant mission state unless authentication, authorization, freshness, and schema checks pass.	Replay, stale-message, revoked-credential, and out-of-scope-command tests.
INV-5: Evidence association	Each consequential gate decision is associated with the input/model/constraint versions and resulting execution status, or explicitly records why evidence is incomplete.	Trace audits, forced logger faults, record-to-command reconciliation.
INV-6: Integrity-chain verification	Modification, deletion, insertion, or reordering of a protected record sequence is detectable under the stated cryptographic assumptions.	Mutation tests over retained logs; key, rollover, truncation, and recovery tests.

5.3 Evidence-chain model

For an ordered record e_t, an abstract integrity chain can be expressed as:

$$h_t = \text{Hash}(h_{t-1} \,\|\, \text{Serialize}(e_t))$$

The record should include the schema version, monotonic or otherwise defensible sequence identity, timestamp source, and references to the model, state, mission, health, and decision versions. The security analysis must explicitly state its threat assumptions—for example, whether the attacker can access storage, compromise the running logger, alter system time, or obtain signing keys. Hash chaining makes some after-the-fact edits evident when verification starts from a trusted anchor; it is not a substitute for trusted capture, key management, redundancy, and retention controls.

6. Research questions and hypotheses

The architecture yields testable hypotheses, but no results can be inferred from the drawings alone. Each hypothesis should be preregistered with operational definitions, scenario distributions, and acceptance criteria before evaluation.

Research question / hypothesis	Independent variable or comparison	Primary outcomes
H1. Deterministic gating reduces hard-constraint violations relative to an otherwise identical planner without the gate.	Full gated pipeline versus simulation-only ablation without the gate.	Violation rate; severity-weighted violations; gate latency; false-reject rate; intervention rate.
H2. Updating constraints before candidate selection improves behavior under predicted component degradation.	Health-conditioned planning versus diagnostic-only or delayed-constraint baselines.	Unsafe/infeasible commands; time from detectable degradation to constraint update; mission utility; unnecessary replans.
H3. Authority-aware reconciliation prevents stale, replayed, or conflicting commands from changing executable mission state.	Reconciliation enabled versus controlled software test harness with reconciliation ablated.	Unauthorized/stale updates accepted; conflict escape rate; detection-to-block latency; availability under benign delay.
H4. Structured evidence records improve decision reconstructability and tamper detection.	Complete logging with integrity protection versus intentionally incomplete or unprotected test records.	Required-field completeness; reconstruction accuracy; tamper-detection rate; logging latency; dropped-record rate.
H5. Cross-layer integration remains within real-time compute and power budgets under concurrent faults.	Nominal load versus progressively stressed CPU/GPU/memory/network/logging conditions.	End-to-end decision latency percentiles; deadline misses; power/thermal margin; gate freshness and fallback rate.

7. Proposed validation protocol

7.1 Staged assurance process

A defensible evaluation should progress through evidence-building stages, with each stage acting as a gate for the next. This order reduces the risk that defects in state handling, asynchronous execution, or fallback behavior are discovered for the first time on a flying vehicle.

- Stage A — Requirements and formal model. Define the operational design domain, hazard analysis, safety constraints, state/authority transition rules, interface timing, fallback policy, and the meaning of every confidence and integrity field. Model-check mission-state transitions and gate semantics where tractable.
- Stage B — Software-in-the-loop simulation. Use a documented aircraft/environment model, seeded scenarios, repeatable sensor error models, and instrumented candidate/gate outputs. Include baseline and ablation variants only within simulation or a controlled test harness.
- Stage C — Software and hardware integration. Exercise scheduling, message ordering, processor contention, timestamp drift, storage exhaustion, power interruptions, cryptographic failures, and interfaces between planner, gate, flight control, and logger.

- Stage D — Hardware-in-the-loop. Use a representative flight-control stack and sensor/actuator interfaces to measure deadline behavior and failure response under constrained conditions. Verify that a rejected or stale command cannot bypass gating through alternate interfaces.
- Stage E — Restricted flight trials. Conduct only after independent review of hazards, fallback, verification evidence, legal permissions, operating conditions, and test-range controls. Begin with low-risk, reversible scenarios and pre-authorized contingency procedures; do not use flight trials to compensate for missing software verification.

7.2 Fault and scenario matrix

Scenario	Injection / stimulus	Expected observable behavior	Evidence to collect
Sensor dropout or stale data	Suppress a modality, delay timestamps, or corrupt calibration/status metadata.	Estimator flags loss/degradation; confidence/provenance reflects changed status; planner/gate receives a valid quality state.	Estimate error, stale-data detection latency, gate outcomes, timestamps and source identity.
Inconsistent sensor observations	Inject bounded and out-of-family disagreement between sensors.	Disagreement is surfaced; policy reduces reliance or selects a conservative response rather than hiding the conflict.	Innovation/residual measures, conflict flags, false alarms, resulting feasible set.
Predicted power/actuator degradation	Inject telemetry trend or fault state before the component reaches a hard limit.	Updated constraint version affects candidate generation or blocks an invalid candidate before control.	Detection lead time, constraint-update latency, infeasible command count, mission utility.
Planner unsafe candidate	Produce a candidate that violates a defined hard constraint in simulation.	Gate rejects or modifies it; any modification is re-evaluated; no unapproved candidate reaches the controller.	Decision reason, violation magnitude, modified trajectory digest, end-to-end latency.
Loss/degradation of communications	Drop packets, vary delay, corrupt integrity, or interrupt the link.	Defined degraded policy activates; unauthenticated or stale state changes do not execute.	Link-state transitions, fallback selection, stale update acceptance, recovery behavior.
Replay/revocation/version conflict	Replay an earlier command; reorder messages; revoke authority; send conflicting versions.	Freshness/authority checks and reconciliation block or resolve the update according to explicit precedence rules.	Conflict state history, accepted/rejected command identity, time to block, false blocking.
Logger overload or power interruption	Exhaust queue/storage or interrupt power during record write.	System exposes incomplete evidence and follows a pre-specified safe logging-fault policy; integrity verification reports gaps.	Dropped-record rate, recoverability, chain verification, added latency and power draw.
Compute saturation / timing fault	Increase load, delay a module, or cause an estimator/planner timeout.	Stale candidates are not executed; safety gate and fallback timing assumptions remain satisfied.	Worst-case/percentile latency, missed deadlines, stale decision rejection, resource usage.

7.3 Metrics and operational definitions

Metric	Definition / measurement rule	Interpretation caution
Hard-constraint violation rate	Number of violations of specified hard constraints per simulated flight-hour or per scenario; also report severity and scenario type.	Zero observed violations do not prove zero risk; report exposure and confidence bounds.
Unauthorized/stale-command acceptance	Count of commands that alter executable mission state despite failed authority or freshness criteria.	Target is zero in the defined test set; coverage of authority combinations must be stated.
Gate intervention and false rejection	Proportion of candidates modified/rejected; false rejection determined against a reference label of feasible candidates.	More interventions may indicate either better protection or poor candidate quality.

Metric	Definition / measurement rule	Interpretation caution
Health response latency	Elapsed time from defined detectable fault criterion to effective planning-constraint version at the relevant gate.	Define detection onset and allowed observation delay before testing.
Decision deadline adherence	End-to-end latency from the agreed input timestamp to gate disposition and control-interface availability.	Report median, p95, p99, worst observed, deadline misses, and hardware context.
Evidence completeness/reconstruction	Fraction of consequential decisions with all required fields; reviewer ability to link inputs, versions, gate result, and execution.	A record can be complete but false; independently compare against ground-truth test harness events.
Tamper detection	Fraction of predefined record modifications, deletions, insertions, truncations, and reorderings detected during verification.	State threat model, trusted anchor, key access, and whether logger compromise is in scope.
Mission utility under faults	Completion of authorized objective subject to constraints; include time, distance/energy, and safe contingency outcomes.	Mission completion must never override a mandatory safety invariant.

7.4 Experimental design and analysis

Use a scenario matrix that spans sensor condition, environment uncertainty, health status, communications condition, mission-state condition, and computing load. Record random seeds, model versions, configurations, scenario labels, and all safety-relevant input/output timestamps. Use a baseline implementation and controlled ablations to estimate which modules contribute to each measured outcome. Ablations that remove safety protections should remain in simulation or an isolated test harness and must not be deployed to an operational aircraft.

For binary outcomes such as unauthorized acceptance or constraint violation, report counts, exposure denominators, and confidence intervals rather than only percentages. For latency and reconstruction time, report distributions and tail percentiles. Determine the required number of trials from a pilot estimate, a predeclared effect of interest, desired statistical power, and the rarity/severity of the target event; do not select the sample size after inspecting results. Stratify by scenario family, report missing data and excluded runs, and use multiplicity control where many primary comparisons are tested. Acceptance thresholds must come from hazard analysis and the intended operational design domain, not from values chosen to make the prototype pass.

8. Failure analysis and design controls

Failure mode	Potential consequence	Proposed control / evidence requirement
Confidence score is uncalibrated or semantically ambiguous.	Planner may over-trust a degraded estimate or compare incomparable scores.	Specify score semantics and calibration domain; test calibration and out-of-domain detection; propagate invalid status.
Planner/gate uses mismatched state or constraint versions.	A candidate appears safe under stale conditions.	Bind candidate, prediction, mission, health, and gate decision to version/digest; reject expired or mismatched approvals.
Safety gate shares an uncontrolled failure mode with planner.	Planner defect or common-mode resource failure disables enforcement.	Analyze independence and common-cause paths; partition critical functions and test overload/failure behavior.
Health predictor misses degradation or updates too late.	Unsafe trajectory remains in candidate set.	Set detection/latency requirements; use conservative defaults for missing health data; fault-inject trends and abrupt failures.

Failure mode	Potential consequence	Proposed control / evidence requirement
Remote authority is authentic but stale/out of scope.	Previously valid commands override current safety or mission intent.	Separate authentication, authorization, freshness, revocation, and semantic validation.
Conflict resolution uses ambiguous precedence.	Two mutually inconsistent mission states may be executed.	Specify deterministic precedence; unresolved states are non-executable; model-check conflict transitions.
Hash chain is correct but initial capture is compromised.	Log accurately preserves incorrect or manipulated data.	Protect capture path and keys; anchor records; add trusted timestamps and independent telemetry where justified.
Logger overload creates silent gaps.	Incident cannot be reconstructed; safety decisions are misrepresented as fully logged.	Mandatory-event policy, explicit gap markers, queue monitoring, storage sizing, and fault response for logging unavailability.
Fallback is not independently validated.	A safety intervention leads to a new hazardous state.	Define fallback envelope and authority; verify transition transient and resource sufficiency under each triggering fault.

9. Discussion

The principal strength of the proposed architecture is cross-layer coupling. A state estimate carries confidence and provenance; health information changes which trajectories are considered feasible; authority affects which mission transitions are permitted; unresolved conflicts explicitly block execution; and the evidence record links inputs, model identity, safety decision, and control outcome. This can make safety assumptions more inspectable than an architecture in which the planner, health monitor, command channel, and logger operate as independent subsystems with weakly specified interfaces.

The most important caveat is that an integrated block diagram is not itself a safety argument. The system may still fail if confidence is poorly calibrated, environment uncertainty is omitted from prediction, the forward model is inaccurate, hard constraints omit relevant hazards, the gate misses a deadline, health updates race with candidate approval, authority keys are compromised, or the fallback is not safe from the current state. A deterministic gate can consistently enforce an incorrectly specified constraint. A well-formed evidence log can faithfully record a flawed decision. The assurance case therefore needs both module-level evidence and evidence for the dependencies and shared failure modes between modules.

The design also creates potential resource trade-offs. Per-field provenance and extensive decision records increase storage and computation; forward evaluation and revalidation may increase latency; conservative health constraints may reduce mission completion; and communication degradation policies may prioritize safety at the cost of availability. The evaluation must report those costs alongside safety outcomes. A successful result should show not simply that the architecture can reject unsafe candidates, but that it does so within a justified timing budget, keeps degraded modes explicit, and preserves sufficient evidence to investigate failures.

The proposed formalization also supports a modular assurance strategy. Some properties—such as rejecting a mission state that lacks authority—may be amenable to finite-state verification. Continuous aircraft dynamics and uncertain environment interactions require other techniques, including simulation, reachability analysis, robust control analysis, and hardware tests. The RTA literature cautions that monitor frequency, switching conditions, and assumptions about the trusted reversionary controller matter to the assurance argument [2,3,6]. Therefore the proposed deterministic safety envelope must be paired with a validated plant/controller interface and a verified fallback, not merely a software condition that returns three labels.

10. Limitations and responsible deployment

This work is limited by the nature of its evidence. The supplied package documents an architecture and patent claims, but does not supply source code, aircraft dynamics, hardware performance, detailed threat models, confidence calibration, safety requirements, defined operating limits, test results, or verification artifacts. The article's equations are candidate formalizations; they are not proof of correctness. The literature cited provides related methods and assurance context, not validation of the proposed system. No inference should be made that the architecture meets any particular airworthiness, operational, security, or regulatory requirement.

Before real-world use, the design would require a system-specific hazard assessment; explicit operating design domain and safety requirements; traceability from hazards to constraints and tests; cyber threat modeling and key management; timing and worst-case resource analysis; verification of the gate and fallback; independent review of test procedures and results; and applicable legal, aviation, and site-specific authorizations. Human supervision should have clearly defined authority and revocation semantics. Failure responses must account for loss of communications, sensor faults, compute failures, invalid mission state, and logger faults without creating a more hazardous transition.

The work is also not a legal opinion, patentability analysis, freedom-to-operate assessment, or filing statement. The source package itself identifies inventor, ownership, public-disclosure, priority, funding, and other legal confirmations as pending. Such questions should be resolved separately by qualified counsel and the relevant responsible parties.

11. Conclusion

This paper formalizes a research agenda for an integrated uncrewed-aircraft architecture that connects confidence-aware estimation, uncertain environment modeling, AI candidate planning, deterministic pre-execution safety gating, health-aware constraint updates, authenticated supervision, mission-state reconciliation, and integrity-protected evidence logging. The key proposition is that safety-critical context should constrain executable behavior and remain traceable across module boundaries. Candidate invariants and a staged evaluation protocol are provided to make that proposition falsifiable. The immediate next step is not to claim that the architecture is safer, but to implement a minimal instrumented prototype, define and review its safety requirements, verify the discrete authority and gating logic, and measure behavior under controlled faults before progressing to hardware-in-the-loop or approved flight trials.

Declarations and reproducibility notes

Data availability. No new empirical dataset was generated for this paper. The architecture was synthesized from the user-provided UCAS patent-application assembly package. A versioned technical implementation, test harness, scenario definitions, and datasets should be published or archived as they become available, subject to applicable intellectual-property, privacy, security, and aviation constraints.

Funding and conflicts of interest. The supplied materials do not contain complete author declarations for funding, financial interests, or competing interests. These statements must be completed by the author(s) before external submission; no absence of conflict is inferred here.

Author contributions. Author identities and contribution roles were not specified in the supplied source package and should be added by the responsible author(s).

Safety and ethics. No flight test or operational deployment is reported. Any follow-on test should be conducted only under the relevant approvals, risk controls, and operating permissions.

References

[1] Hall, D. L., & Llinas, J. (1997). An introduction to multisensor data fusion. Proceedings of the IEEE, 85(1), 6–23. https://doi.org/10.1109/5.554205

[2] Schierman, J. D., DeVore, M. D., Richards, N. D., & Clark, M. A. (2020). Runtime assurance for autonomous aerospace systems. Journal of Guidance, Control, and Dynamics. https://doi.org/10.2514/1.G004862

[3] Slagel, J. T., White, L. M., Dutle, A., Muñoz, C. A., & Crespo, N. (2024). A verification framework for runtime assurance of autonomous UAS. Proceedings of the 43rd Digital Avionics Systems Conference. NASA Technical Reports Server, Document ID 20240007986. https://ntrs.nasa.gov/citations/20240007986

[4] Ancel, E., Moore, A. J., Young, S. D., Dill, E. T., Quach, C. P., & Smalling, K. M. (2024). Testing a run-time assurance framework coupled with integrated risk mitigation capabilities for autonomous urban UAS flights. Proceedings of the 43rd Digital Avionics Systems Conference. NASA Technical Reports Server, Document ID 20240007663. https://ntrs.nasa.gov/citations/20240007663

[5] Hsu, K.-C., Hu, H., & Fisac, J. F. (2024). The safety filter: A unified view of safety-critical control in autonomous systems. Annual Review of Control, Robotics, and Autonomous Systems, 7, 47–72. https://doi.org/10.1146/annurev-control-071723-102940

[6] ASTM International. (2021). ASTM F3269-21: Standard practice for methods to safely bound behavior of aircraft systems containing complex functions using run-time assurance. https://store.astm.org/f3269-21.html

[7] National Institute of Standards and Technology. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

[8] Regenscheid, A. (2018). Platform firmware resiliency guidelines (NIST Special Publication 800-193). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-193

[9] Scarfone, K., & Souppaya, M. (2006). Guide to computer security log management (NIST Special Publication 800-92). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-92

Source-package traceability note. Architecture terminology and depicted module relationships are derived from the supplied UCAS_FINAL_Patent_Application_Package.zip, particularly the consolidated specification, invention disclosure memorandum, and corrected FIGS. 4–5. The research formalization and evaluation protocol are proposed additions for technical development and review.